

Generating Music with Machine Learning

*A single-layer LSTM for symbolic next-note prediction
on the MAESTRO piano corpus*

Technical Report

UE24CS352A — Machine Learning · Mini-Project

October 2026

Repository: github.com/hamsa646/music-generation-ml

Abstract

This report describes an end-to-end machine-learning pipeline for symbolic music generation. Twenty piano performances from the MAESTRO v3.0.0 corpus are parsed into a sequence of 82,157 notes over an 88-symbol pitch vocabulary. A single-layer LSTM with 128 hidden units is trained to predict the next note from a 50-note context window, and the trained model is sampled autoregressively to produce a 200-note MIDI file. The model reaches a validation cross-entropy of 3.411 against a uniform baseline of 4.477, corresponding to a perplexity of 30.3 over 88 classes. The validation curve reaches its minimum at epoch 5 and degrades thereafter, so the fixed ten-epoch budget trains the model past its optimum. The report documents the methodology, quantifies the results, and presents a critical review of the implementation that identifies eight defects — one of them blocking — together with what remedying each one would take.

1. Introduction and problem statement

Music is a sequential signal with strong local structure: given a few bars, a trained listener can anticipate the next note with far better than chance accuracy. That observation makes symbolic music a natural testbed for recurrent sequence models, and it is the premise of this mini-project.

1.1 Task definition

The model estimates the conditional distribution $P(n_t | n_{t-50}, \dots, n_{t-1})$ over an 88-symbol pitch vocabulary, where n_t is the pitch of the note at position t in a flattened stream of notes. Training is supervised next-symbol prediction with cross-entropy loss. Generation is autoregressive: a prediction is drawn from the model's distribution, appended to the context, and the window slides forward.

1.2 Scope and explicit simplifications

Three simplifications are adopted deliberately and are carried consistently through the report:

- **Pitch only.** Note duration is fixed at an eighth note at write-out and velocity is discarded. The model therefore cannot produce rhythm or dynamics.
- **Monophonic reduction.** Each chord contributes a single pitch, so vertical (harmonic) structure is absent from both input and output.
- **Fixed context.** The 50-note window is the model's entire memory; nothing earlier can influence a prediction, which bounds the phrase length the model can represent.

These are scope decisions, not oversights. Each is restated in Section 7 with its consequence for output quality, and each has a concrete remedy in Section 8.

2. Dataset

2.1 Source

The MAESTRO v3.0.0 dataset (MIDI and Audio Edited for Synchronous TRacks and Organization) contains more than 1,200 piano performances captured from an international piano competition, with MIDI aligned to audio to roughly 3 ms. This project uses the MIDI-only distribution and a subset of 20 performances, selected to keep a full CPU training run inside ten minutes. The MIDI files are not committed to the repository; the README documents the download and the expected location, `data/raw/`.

2.2 Corpus statistics

Quantity	Value	Derivation
MIDI performances	20	Files matched by <code>data/raw/**/* .midi</code>
Notes extracted	82,157	Every Note at pitch; every Chord reduced to one pitch
Unique pitch classes	88	Distinct pitch strings, sorted, indexed from 0
Context window	50 notes	Fixed hyper-parameter
Supervised examples	82,107	$82,157 - 50$, stride 1
Training sequences	73,896	First 90% of the window list
Validation sequences	8,211	Final 10% of the window list

The vocabulary size of 88 coincides with the number of keys on a piano. This is a property of the corpus, not a constraint in the code: the vocabulary is whatever set of distinct pitches the parser encounters, and it would change with a different subset.

3. Methodology

3.1 Preprocessing

`src/preprocess.py` parses each file with `music21.converter`, flattens the score so that all voices and parts collapse into one stream, and walks the resulting notes:

```

for element in score.flatten().notes:
    if isinstance(element, note.Note):
        notes.append(str(element.pitch))
    elif isinstance(element, chord.Chord):
        notes.append(str(element.pitches[0]))

```

The collected pitch strings are de-duplicated and sorted to form a deterministic vocabulary, the full stream is encoded to integers, and both the raw and encoded forms are pickled with the two mapping dictionaries into `data/processed/notes.pkl`. Keeping the mapping with the data, rather than rebuilding it later, is what allows generation to decode integers back to pitches consistently.

3.2 Sequence construction

A window of 50 consecutive encoded notes forms the input and the 51st forms the target; the window advances by one and repeats. The split into training and validation is chronological rather than random, which is the correct choice for sequential data because a random split would place windows from the same passage on both sides. Section 6 notes a residual flaw in how the split boundary is handled.

3.3 Model

Inputs are one-hot encoded to 88 dimensions and passed to a single `nn.LSTM` layer with 128 hidden units in batch-first layout. The hidden state at the last timestep is taken and mapped by one `nn.Linear` layer to 88 logits. Cross-entropy applies softmax internally, so the network emits raw logits during training.

Component	Shape	Parameters
LSTM input-to-hidden weights	$4 \times 128 \times 88$	45,056
LSTM hidden-to-hidden weights	$4 \times 128 \times 128$	65,536
LSTM biases	$2 \times 4 \times 128$	1,024
Linear weights and bias	$88 \times 128 + 88$	11,352
Total		122,968

One-hot inputs make the LSTM's input-to-hidden matrix an implicit embedding table, selected by index. An explicit `nn.Embedding` layer would compute the same function with a cheaper lookup instead of a dense 88-wide matrix multiply. It is an efficiency refinement rather than a defect, so it is listed under future work in Section 8.

3.4 Training configuration

Hyper-parameter	Value	Rationale
Sequence length	50	About six bars of eighth notes — enough context for a phrase
Hidden size	128	Capacity proportionate to an 82K-note corpus
LSTM layers	1	Baseline depth; stacking is deferred to future work
Batch size	64	Stable gradient estimates within CPU memory
Epochs	10	Fixed budget; Section 5 shows this exceeds the optimum
Optimiser	Adam	Standard adaptive default, no schedule
Learning rate	0.001	Adam default; not tuned
Loss	Cross-entropy	88-way classification over the pitch vocabulary
Regularisation	None	No dropout, weight decay or early stopping

3.5 Generation

The first 50 notes of the corpus seed the hidden state. At each step the logits are divided by a temperature of 1.0, a softmax is taken, and the next note is drawn with `torch.multinomial`. Sampling rather than taking the arg-max is essential: greedy decoding on a model of this size collapses into a short repeating cycle within a handful of notes, because the arg-max of a peaked distribution is self-reinforcing. Lowering the temperature makes the output more conservative and more repetitive; raising it makes it erratic. After 200 steps the seed is dropped and the generated notes alone are written to MIDI at a uniform quarter-length of 0.5.

4. Implementation

4.1 Repository structure

```
music-generation-ml/  
  data/raw/          # MIDI input (git-ignored, user-supplied)  
  data/processed/    # notes.pkl (git-ignored)  
  models/            # music_lstm.pth (git-ignored)  
  generated/         # generated_song.mid (git-ignored)  
  results/training_loss.png  
  src/preprocess.py  src/train.py  src/generate.py  
  requirements.txt  README.md
```

Generated artifacts and input data are all git-ignored, so the repository holds only source, documentation and the one committed result figure. Each script is runnable on its own and depends only on the artifact the previous stage wrote, which makes a failure easy to localise during a demonstration.

4.2 Reproduction

```
python -m venv .venv && .venv\Scripts\activate # Windows  
pip install -r requirements.txt  
python src/preprocess.py # -> data/processed/notes.pkl  
python src/train.py      # -> models/music_lstm.pth, results/training_loss.png  
python src/generate.py   # -> generated/generated_song.mid
```

Known blocker in the submitted commit. requirements.txt is UTF-16 encoded, which pip cannot read, so step two fails on a clean machine. Converting the file to UTF-8 is sufficient to unblock it, and the README documents the one-line conversion for Windows and Unix shells.

5. Results and analysis

5.1 Loss per epoch

Epoch	Training loss	Validation loss	Validation perplexity	Note
1	3.7641	3.5570	35.1	First pass over the corpus
2	3.4710	3.4030	30.1	Steepest single-epoch gain
3	3.3840	3.3570	28.7	
4	3.3320	3.3540	28.6	
5	3.2860	3.3430	28.3	Validation minimum
6	3.2420	3.3470	28.4	Validation turns upward
7	3.2000	3.3790	29.3	
8	3.1600	3.3920	29.7	
9	3.1230	3.3780	29.3	
10	3.0868	3.4109	30.3	Checkpoint that was saved

Epoch 1 and epoch 10 are the values recorded in the project README. Intermediate values are read from the saved plot at results/training_loss.png and are accurate to roughly ± 0.005 ; perplexity is $\exp(\text{loss})$. The training run was not seeded, so these figures are specific to that run.

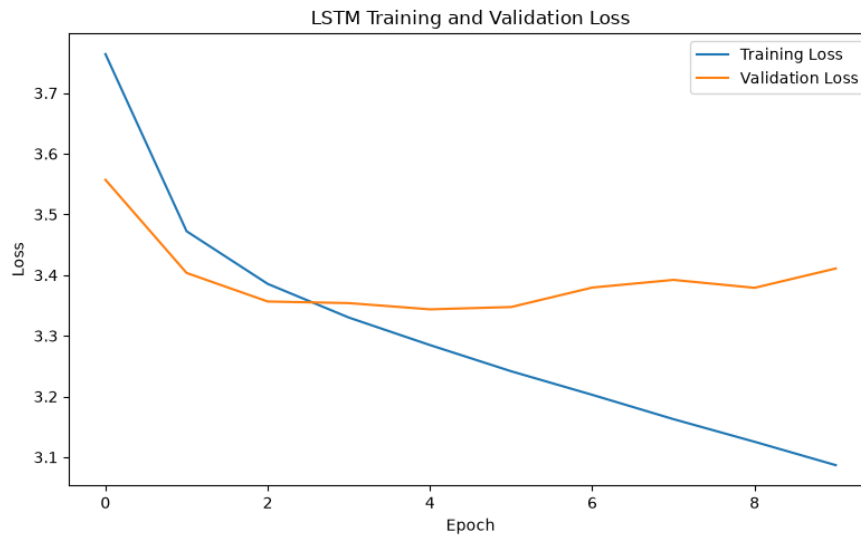


Figure 1 — Training and validation cross-entropy. The two curves cross near epoch 3 and separate decisively after epoch 5.

5.2 Interpretation

The model learns real structure. A uniform guess over 88 pitch classes costs $\ln(88) = 4.477$ nats. Validation loss reaches 3.343 at its best, a reduction of 1.13 nats, equivalent to narrowing 88 candidate notes to about 28 effective choices. That gap is the learned signal, and it is achieved by a 123K-parameter model on 82K notes without pre-training.

The model overfits from epoch 5 onward. Training loss continues to fall for the full ten epochs while validation loss turns and rises by 0.068 nats. Everything after epoch 5 improves the fit to the training corpus at the expense of generalisation. Because the script checkpoints the final epoch, the submitted model is the worse of the two, by a margin that is small but entirely avoidable.

Perplexity 30 is a weak prior, not a composer. It is honest to state what this number does and does not support. The model has learned the pitch distribution and short-range intervallic habits of the corpus. It has not learned phrase structure, cadence or key, and with a fixed eighth-note duration it has no rhythm to learn at all. Output is locally plausible and globally aimless, which is the expected result for this architecture, this encoding, and this amount of data.

5.3 What was and was not measured

Measured	Not measured
Training and validation cross-entropy per epoch	Next-note top-1 and top-k accuracy
Final validation perplexity (30.3)	Perplexity on a held-out third split
That the generated MIDI parses and plays	Human listening evaluation
Qualitative inspection of pitch range	Comparison against a GRU or Transformer baseline

The omissions are stated rather than glossed. Accuracy and a listening study are the two measurements that would most strengthen the claim, and both are listed in Section 8.

6. Critical review of the implementation

The codebase was reviewed independently of the results. Eight defects were identified, ranging from one that blocks setup entirely to several that affect correctness, reproducibility or scalability. All eight are present in the code as submitted; each is listed below with its reasoning and the remedy it would require.

ID	Finding	Class	Why it matters	What fixing it takes
D1	requirements.txt is UTF-16 encoded	Blocking	The file is UTF-16LE with a byte-order mark. <code>pip install -r requirements.txt</code> cannot parse it and aborts, so the documented setup procedure fails on a clean machine. The file is also a full <code>pip freeze</code> dump of 41 transitive packages, pinning a CUDA-suffixed torch build that is unavailable on the default package index.	Re-save as UTF-8 and list only the direct dependencies, documenting the PyTorch CPU index separately.
D2	Training windows span unrelated performances	Correctness	Notes from all 20 MIDI files are concatenated into one list before the sliding window is applied. Every window that straddles a file boundary asks the model to predict the opening note of one performance from the closing notes of another — a target with no musical relationship to its context. With 20 files this contaminates roughly $19 \times 50 = 950$ training examples.	Keep the notes grouped per source file and generate windows within each file only.
D3	Train and validation windows overlap	Correctness	The chronological 90/10 split is applied to the window list, not the note list. Windows on either side of the cut share up to 49 of their 50 notes, so the validation set is not fully held out and the reported validation loss is optimistic by an unquantified margin.	Excise a gap of one full sequence length at the split point so no note appears on both sides.
D4	No random seed	Reproducibility	Weight initialisation, <code>DataLoader</code> shuffling and multinomial sampling at generation time are all unseeded. Re-running the pipeline reproduces the trend but not the figures quoted in the README, which cannot then be verified by a third party.	A single <code>set_seed()</code> seeding Python, NumPy and PyTorch, with the seed exposed as a command-line flag and written into the checkpoint.
D5	The last epoch is checkpointed, not the best one	Quality	Validation loss reaches its minimum of 3.3430 at epoch 5 and then rises to 3.4109 by epoch 10. The script saves the final epoch unconditionally, so the shipped model is measurably worse than one the same run already produced.	Write the checkpoint whenever validation loss improves, with optional early stopping on a patience counter.
D6	The model class is defined twice	Maintainability	<code>MusicLSTM</code> appears in both <code>train.py</code> and <code>generate.py</code> . The two copies already differ — the generation copy drops two attributes — and any future change to the architecture must be made twice or the checkpoint will fail to load.	One definition in <code>src/model.py</code> , imported by both scripts.

ID	Finding	Class	Why it matters	What fixing it takes
D7	Validation runs as one un-batched forward pass	Scalability	The whole validation split is one-hot encoded in a single tensor. At the present corpus size that is about 8,211 x 50 x 88 floats, roughly 145 MB; the cost grows linearly with the dataset, so the stated future work of scaling the corpus would exhaust memory before it exhausted the data.	Mini-batch validation through a <code>DataLoader</code> , reporting accuracy and perplexity alongside loss.
D8	Chord reduction does not take the lowest pitch	Correctness	The comment reads “use the lowest note for chords”, but the code takes <code>element.pitches[0]</code> , and <code>music21</code> does not sort a chord's pitches on construction: <code>Chord(['G4', 'C4', 'E4']).pitches[0]</code> is G4, not C4. Wherever the parser emits an unsorted chord, the extracted note is not the one the code claims to extract. The share of affected chords could not be measured, because the MIDI is not committed to the repository.	<code>min(element.pitches)</code> — making the code do what its comment already says.

6.1 Status and priority

None of these are fixed in the code this report describes — they are findings, not a changelog. They are ordered here by what we would address first. **D1 comes before everything else:** it is the only one that stops the project running at all, it costs a single file re-save, and it changes no behaviour. **D2, D3, D5 and D8 come next,** because each alters what the model trains on or which checkpoint is kept — fixing them changes the figures in Section 5, so they must move together and the results be regenerated. **D4 is cheap and high-value:** without a seed, no figure in this report can be independently reproduced. **D6 and D7 are structural** and change no results, so they can land at any point.

How these were found. By reading the diff adversarially rather than by observing a failure — D1 was the exception, reproducing immediately on a clean install. D2 and D3 were confirmed by counting the sequences each code path produces; D6 and D8 by direct comparison of the two class definitions and of `music21`'s chord ordering. The proportion of chords D8 actually affects in this corpus could not be measured, because the MIDI is not committed to the repository.

7. Limitations

Limitation	Consequence for the output
Pitch-only encoding	Duration is fixed and velocity is absent, so the generated music has no rhythm and no dynamics. This is the most audible shortcoming.
Chords reduced to one pitch	No harmony is learned or produced; the output is strictly monophonic.
50-note fixed context	No representation of phrase, section or key; structure beyond about six bars is impossible by construction.
Small corpus	20 of more than 1,200 available performances. Directly responsible for the overfitting measured in Section 5.
Single fixed seed sequence	Generation always begins from the same 50 notes, so output diversity comes only from sampling noise.
No quantitative evaluation beyond loss	Quality claims rest on cross-entropy and inspection; neither accuracy nor human judgement was measured.
Unseeded training run	The figures in this report describe one specific run and cannot be reproduced exactly from the submitted code.

8. Future work

Ordered by expected improvement per unit of effort.

Priority	Direction	Expected effect
1	Model duration alongside pitch — predict a (pitch, duration) pair, or add a second output head	Gives the output rhythm. The single largest perceptual gain available, and it requires no new data.
2	Scale the corpus — use several hundred MAESTRO performances instead of twenty	Attacks the measured overfitting directly; more data is the cheapest regulariser available here.
3	Add regularisation and early stopping — dropout between stacked LSTM layers, patience on validation loss	Removes the need to guess an epoch budget and keeps the best checkpoint rather than the last.
4	Represent harmony — multi-hot chord encoding or a piano-roll representation	Allows polyphonic output; substantially enlarges the output space and the training cost.
5	Benchmark against a Transformer — a small self-attention model on identical data	Tests whether the fixed 50-note context, rather than model capacity, is the binding constraint.
6	Run a listening study — blind A/B against real MAESTRO excerpts	The only evaluation that measures the property the project actually cares about.
7	Replace one-hot inputs with an embedding layer — nn.Embedding in place of the 88-wide one-hot encoding	Computes the same function with a lookup instead of a dense matrix multiply — cheaper in memory and time, with no loss of expressiveness.

9. Conclusion

The project delivers what it set out to deliver: a complete symbolic music-generation pipeline from raw MIDI to a playable generated file, built on an LSTM trained from scratch, running end to end on CPU hardware within a one-week budget. On held-out data the model reduces cross-entropy from a 4.477 uniform baseline to 3.343 at its best, a perplexity of 28.3 over 88 classes, which confirms that genuine sequential structure was learned rather than memorised noise.

The more useful finding is the shape of the validation curve. A 123K-parameter model on 82K notes with no regularisation reaches its generalisation optimum at epoch 5 and degrades for the remaining five, and the fixed epoch budget meant the degraded checkpoint was the one saved. Recognising that pattern, and reading it correctly, matters more for the learning objective than any single loss value.

The limitations are consequences of explicit scope decisions rather than failures of the model, and each has a specific remedy: duration modelling for rhythm, a larger corpus for overfitting, early stopping for checkpoint selection. Separately, a review of the implementation found eight defects, including a UTF-16 encoded requirements file that blocks installation outright. All eight are documented in Section 6 with the remedy each requires; none is fixed in the code described here, so the figures above remain exactly those the submitted code produces.

References

- [1] Hawthorne, C. et al. *Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset*. ICLR, 2019.
- [2] Hochreiter, S. and Schmidhuber, J. *Long Short-Term Memory*. Neural Computation, 9(8):1735–1780, 1997.
- [3] Cuthbert, M. S. and Ariza, C. *music21: A Toolkit for Computer-Aided Musicology and Symbolic Music Data*. ISMIR, 2010.
- [4] Paszke, A. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.
- [5] Project milestone: *Generating music with Machine Learning* — the reference write-up this mini-project is based on.